

Definiții – Limbaje formale, automate și compilatoare

1. Structura unui compilator

Compilatorul este un program care traduce un program scris într-un limbaj de nivel înalt într-un program obiect echivalent, de obicei în limbaj mașină, și poate produce un fișier de erori.

Programul sursă este programul primit la intrare de compilator.

Programul obiect este programul echivalent produs la ieșire de compilator: executabil, translatabil, în limbaj mașină sau într-o formă apropiată de acesta.

Analiza lexicală este faza care grupează caracterele programului sursă în atomi lexicali: cuvinte cheie, operatori, constante și identificatori.

Analiza sintactică este faza care preia șirul de atomi lexicali și identifică structuri sintactice: expresii, liste, instrucțiuni și proceduri.

Analiza semantică este faza care îmbogățește arborele sintactic cu informații suplimentare, completează tabela de simboluri și verifică tipurile.

Generarea codului intermediar este faza în care programul este tradus într-o formă intermediară, apropiată de limbajul de asamblare.

Optimizarea codului este faza în care se elimină redundanțele, calculele inutile și variabilele inutile, pentru o execuție mai eficientă.

Generarea codului obiect este faza în care codul intermediar este translatat într-o secvență de instrucțiuni mașină sau într-o formă apropiată de acestea.

Gestiunea tabeli de simboluri este funcția prin care compilatorul înregistrează identificatorii și informațiile asociate acestora.

Tratarea erorilor este colecția de proceduri activate când se detectează o greșală în program.

2. Gramatici

Alfabetul este o mulțime finită și nevidă de simboluri.

Cuvântul este o secvență finită de simboluri dintr-un alfabet.

Cuvântul vid este cuvântul de lungime zero și se notează cu ϵ .

Limbajul formal este orice submulțime $L \subseteq V^*$, unde V^* este mulțimea tuturor cuvintelor peste alfabetul V .

Gramatica este un sistem $G = (N, \Sigma, P, S)$, unde:

- N este alfabetul neterminalelor;
- Σ este alfabetul terminalelor;
- P este mulțimea regulilor de producție;
- $S \in N$ este simbolul inițial.

Regula de producție este o regulă de forma $u \rightarrow v$, care indică faptul că secvența u poate fi înlocuită cu secvența v .

Derivarea directă este relația $w \Rightarrow \omega$, unde $w = \alpha\upsilon\beta$, $\omega = \alpha v\beta$ și există producția $u \rightarrow v$ în gramatica G .

Derivarea este aplicarea succesivă a uneia sau mai multor derivări directe.

Derivarea în zero sau mai mulți pași se notează cu $\Rightarrow^*$ și reprezintă închiderea reflexivă și tranzitivă a relației de derivare directă.

Limbajul generat de o gramatică este mulțimea:

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}.$$

Gramaticile echivalente sunt gramaticile care generează același limbaj.

3. Clasificarea Chomsky

Gramatica de tip 0 este gramatica fără restricții impuse asupra producțiilor.

Gramatica de tip 1, numită și gramatică dependentă de context, este gramatica în care orice producție $u \rightarrow v$ satisface condiția $|u| \leq |v|$.

Gramatica de tip 2, numită și gramatică independentă de context, este gramatica în care orice producție are în partea stângă un singur neterminalel.

Gramatica de tip 3, numită și gramatică regulată, este gramatica în care producțiile sunt restricționate astfel încât partea dreaptă are formă regulată.

Ierarhia Chomsky este relația dintre familiile de limbaje:

$$L_3 \subset L_2 \subset L_1 \subset L_0.$$

4. Recursivitatea limbajelor

Limbajul recursiv este un limbaj L peste alfabetul Σ pentru care există un algoritm care, pentru orice secvență $w \in \Sigma^*$, determină dacă $w \in L$ sau $w \notin L$.

5. Automate finite

Automatul finit este un sistem $A = (Q, \Sigma, \delta, q_0, F)$, unde:

- Q este mulțimea stărilor automatului;
- Σ este alfabetul de intrare;
- $\delta : Q \times \Sigma \rightarrow P(Q)$ este funcția de tranziție;
- q_0 este starea inițială;
- F este mulțimea stărilor finale.

Configurația unui automat finit este o pereche (q, α) , unde q este starea curentă, iar α este secvența de pe banda de intrare rămasă de citit.

Configurația inițială este (q_0, w) , unde w este cuvântul de intrare.

Configurația finală este (q, ε) , unde $q \in F$.

Trecerea unui automat finit este un pas de la configurația $(q, a\alpha)$ la configurația (p, α) , dacă $p \in \delta(q, a)$.

Limbajul acceptat de un automat finit este mulțimea cuvintelor $w \in \Sigma^*$ pentru care automatul ajunge din configurația inițială (q_0, w) într-o configurație finală (p, ε) , cu $p \in F$.

Automatul finit determinist este automatul în care, pentru fiecare stare și simbol de intrare, există cel mult o stare următoare.

Automatul finit complet determinist este automatul în care, pentru fiecare stare și simbol de intrare, există exact o stare următoare.

Automatul finit nedeterminist este automatul care nu respectă condiția de determinism.

6. Automate pushdown

Automatul pushdown este un sistem $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, unde:

- Q este mulțimea stărilor automatului;

- Σ este alfabetul de intrare;
- Γ este alfabetul intern, adică alfabetul stivei pushdown;
- δ este funcția de tranziție;
- q_0 este starea inițială;
- Z_0 este simbolul intern inițial;
- F este mulțimea stărilor finale.

Configurația unui automat pushdown este un triplet (q, w, γ) , unde q este starea curentă, w este secvența de intrare rămasă de citit, iar γ este conținutul stivei pushdown.

Configurația inițială este (q_0, w, Z_0) .

Trecerea unui automat pushdown este un pas de la $(q, aw, Z\gamma)$ la $(q_1, w, \alpha\gamma)$, dacă $(q_1, \alpha) \in \delta(q, a, Z)$.

Limbajul acceptat cu stiva vidă este mulțimea cuvintelor w pentru care automatul ajunge din (q_0, w, Z_0) într-o configurație de forma $(p, \varepsilon, \varepsilon)$.

Limbajul acceptat după criteriul stării finale este mulțimea cuvintelor w pentru care automatul ajunge din (q_0, w, Z_0) într-o configurație de forma (p, ε, α) , unde $p \in F$.

Automatul pushdown determinist este automatul pushdown în care, pentru o configurație dată, alegerea tranziției este unică.

7. Gramatici LL

Gramatica $LL(k)$ este o gramatică independentă de context în care producția folosită pentru expandarea neterminalului cel mai din stânga poate fi determinată unic cunoscând partea deja citită din cuvânt, următoarele k simboluri de la intrare și neterminalul care trebuie expandat.

Gramatica LL este o gramatică $LL(k)$ pentru un anumit k .

Funcția $PRIM_k(\alpha)$ este mulțimea prefixelor terminale de lungime cel mult k care pot apărea la începutul cuvintelor derivate din secvența α .

Funcția $URM_k(\beta)$ este mulțimea șirurilor terminale care pot urma după secvența β într-o formă derivată din simbolul inițial.

8. Gramatici LR

Gramatica LR(k) este o gramatică independentă de context pentru care analiza se face de la stânga la dreapta, construind invers o derivare la dreapta și folosind k simboluri de anticipare.

Analiza LR(k) este analiza în care, pe baza prefixului viabil și a următoarelor k simboluri de intrare, se determină unic acțiunea de analiză: trecere, reducere, acceptare sau eroare.

Prefixul viabil este un prefix al părții drepte a unei forme sentențiale care poate apărea pe stiva unui analizor LR.

Linia LR(k) este un element de forma $[A \rightarrow \beta_1 \dots \beta_2, u]$, unde punctul marchează poziția curentă în producție, iar u este șirul de anticipare.

9. Analiza sintactică

Analiza sintactică a unei secvențe $\alpha \in L(G)$ este efectuată atunci când se găsește cel puțin un arbore generator pentru α .

Analiza sintactică la stânga este analiza care corespunde unei derivări în care se expandează mereu neterminalul cel mai din stânga.

Analiza sintactică la dreapta este analiza care corespunde unei derivări în care se expandează mereu neterminalul cel mai din dreapta.

Analiza sintactică descendentă este analiza care construiește arborele de derivare de la rădăcină spre frunze.

Principiul analizei descendente constă în expandarea unui neterminal din vârful stivei cu partea dreaptă a unei producții și în ștergerea unui terminal din vârful stivei când acesta coincide cu terminalul curent de la intrare.

Analiza sintactică ascendentă este analiza care construiește arborele de derivare de la frunze spre rădăcină, prin reduceri succesive.

Principiul analizei ascendente constă în identificarea unei secvențe care este partea dreaptă a unei producții și reducerea ei la neterminalul din partea stângă a producției, până se obține simbolul inițial.

10. Cod intermediar

Codul intermediar este reprezentarea programului într-un limbaj intermediar, obținută după analiza sintactică și semantică.

Limbajul intermediar este mai apropiat de limbajul de asamblare decât de limbajul sursă.

Instrucțiunea de cod intermediar este o operație împreună cu operanzii asociați.

Operațiile din codul intermediar sunt, în general, operații aritmetice, atribuiri, teste și salturi.

Operanzii din codul intermediar sunt referințe la intrări din tabela de simboluri, nu regiștri sau cuvinte de memorie.

Formele codului intermediar sunt: forma poloneză, arborii sintactici, tripletele și cuadrupele.

Codul cu trei adrese este o formă de cod intermediar în care instrucțiunile au de obicei cel mult trei adrese: două pentru operanzi și una pentru rezultat.

11. Cod obiect

Codul obiect este forma produsă în ultima fază a compilării, reprezentând un program executabil sau aproape executabil.

Programul executabil este forma de cod obiect care poate fi executată direct.

Programul obiect translatabil este forma de cod obiect care necesită editare de legături înainte de execuție.

Programul în limbaj de asamblare este forma de cod obiect formată din instrucțiuni simbolice, care necesită asamblare înainte de execuție.

Programul într-un alt limbaj este forma de cod obiect care necesită cel puțin o compilare suplimentară pentru a fi executată.

Generarea codului obiect este procesul prin care se parcurge forma intermediară a programului și se produc instrucțiuni corespunzătoare calculatorului țintă.

12. Tabela de simboluri

Tabela de simboluri este structura în care compilatorul concentrează informația despre numele simbolice care apar în programul sursă.

Intrarea în tabela de simboluri este înregistrarea asociată unui identificator.

Numele este șirul de caractere care reprezintă identificatorul.

Informația asociată identificatorului conține atribute precum tipul, utilizarea, domeniul de definiție și adresa de memorie.

Atributul unui identificator este o informație asociată numelui simbolic, cum ar fi tipul, adresa, rolul sau domeniul de valabilitate.

Tabela de simboluri neordonată este tabela în care intrările sunt adăugate în ordinea apariției.

Tabela de simboluri ordonată alfabetic este tabela în care intrările sunt păstrate în ordinea alfabetică a numelor.

Tabela de simboluri arborescentă este tabela reprezentată printr-un arbore, de obicei arbore binar de căutare.

Tabela de simboluri dispersată este tabela organizată printr-o funcție de dispersie care împarte identificatorii în clase.

Domeniul de valabilitate al unei variabile este zona programului în care identificatorul desemnează acea variabilă.

Blocul activ este blocul al cărui sfârșit nu a fost încă întâlnit în timpul analizei.